UNIVERSIDAD DON BOSCO

Escuela de Computación

MANUAL TÉCNICO

Sistema de Administración y Reserva de Habitaciones de Hotel

API REST con Java y Spring Boot

Fase Final del Proyecto

Asignatura: DWF G02T

Docente: Ing. Mario Alvarado

Fecha de entrega: 29 de octubre de 2026

Integrantes del equipo

Roberto Antonio Cabrera Navas CN233178

William Aaron Peralta Cruz PC210574

Delmy Beatriz González Aragón GA252427

Helen Verónica Portillo Rodríguez PR230742

Usuario de GitHub: Dark-Side-Ghost

Repositorio: https://github.com/Dark-Side-Ghost/DFW-Sistema-Hotelero.git

1. Introducción

Este documento es el manual técnico del sistema de administración y reserva de habitaciones de una cadena hotelera. El sistema se implementó como una API REST [1] desarrollada con Java y Spring Boot [2], que es consumida por un frontend web construido con React [3]. Permite gestionar hoteles, tipos de habitación, habitaciones, servicios, políticas de cancelación, reservas, pagos, reclamos y estadísticas, con autenticación mediante JSON Web Tokens (JWT) [4] y control de acceso por roles. Los datos se almacenan en una base de datos relacional MySQL [5].

El manual describe los objetivos, los módulos del sistema, las herramientas y dependencias utilizadas, el modelo relacional de la base de datos, la estructura del proyecto, los mecanismos de seguridad y validación, los endpoints principales, el frontend, los usuarios predefinidos para realizar pruebas y los pasos para configurar y ejecutar el proyecto. Las referencias bibliográficas se encuentran al final del documento en formato IEEE.

2. Objetivos

2.1 Objetivo general

Desarrollar una API REST segura y funcional que permita gestionar el proceso completo de reserva de habitaciones de una cadena hotelera, junto con un frontend que la consuma.

2.2 Objetivos específicos

- Implementar operaciones CRUD sobre los módulos del sistema con persistencia en una base de datos relacional MySQL.
- Proteger los endpoints con Spring Security y JWT, con perfiles de acceso diferenciados por rol.
- Validar en el servidor los datos de entrada: campos obligatorios, formatos mediante expresiones regulares y reglas de coherencia como fechas y capacidades.
- Implementar el proceso de reserva: búsqueda de disponibilidad, registro del huésped, pago, modificación y cancelación con cargos o reembolsos, además del check-in y el check-out.
- Documentar la API con Swagger/OpenAPI y mostrar mensajes de error y de éxito claros al usuario.
- Construir un frontend en React que haga funcionales los prototipos de la Fase 1 y consuma la API.

3. Descripción general de los módulos

Módulo	Descripción
Autenticación y usuarios	Inicio de sesión con JWT, registro de clientes y administración de usuarios (crear, actualizar, eliminar, activar y desactivar) a cargo del administrador.
Hoteles	Registro y gestión de hoteles y sucursales (ciudad, país, categoría, dirección y teléfono).
Tipos de habitación y habitaciones	Catálogo de tipos con su capacidad máxima y habitaciones de cada hotel con precio base por noche, estado y servicios incluidos.
Servicios	Servicios incluidos en las habitaciones y servicios adicionales (alimentación, transporte, limpieza y otros).
Políticas de cancelación	Días previos límite y porcentaje de cargo por hotel, usados para calcular cargos y reembolsos.
Búsqueda y disponibilidad	Consulta por ciudad, fechas, número de huéspedes y tipo de habitación opcional, con precio por noche y precio total de la estadía.
Reservas, huéspedes y pagos	Registro del huésped, creación de la reserva con su resumen, pago (método, monto, fecha y estados), modificación, cancelación, check-in y check-out.
Reclamos	Registro y seguimiento de reclamos y solicitudes de los huéspedes.
Estadísticas	Ocupación, reservaciones, cancelaciones e ingresos.

Roles del sistema: ADMIN (gestiona todas las funcionalidades y los usuarios), RECEPCIONISTA (gestión operativa: reservas del día, pagos, check-in/check-out, cancelaciones y reclamos) y CLIENTE (huésped que busca, reserva, paga, modifica y cancela sus reservas y reporta reclamos). Un cliente solo puede ver y modificar sus propias reservas.

Nota de diseño: la tarifa se modela como el precio base por noche de cada habitación (habitaciones.precio_base_noche) y como el precio de los servicios; las condiciones económicas de cancelación se definen en politicas_cancelacion. El huésped

y el pago se registran dentro del flujo de la reserva.

3.1 Proceso de reserva y estados

El cliente busca habitaciones disponibles, elige una, registra los datos del huésped y los servicios adicionales, revisa el resumen y paga. El servidor calcula el precio total (noches × tarifa + servicios), verifica que no existan reservas que se crucen en esas fechas y crea la reserva. Con pago por tarjeta o transferencia la reserva queda confirmada; si el cliente elige efectivo, el pago queda pendiente hasta que recepción lo registre. Recepción realiza el check-in y el check-out. Si se cancela, el reembolso se calcula con la política del hotel: sin cargo cuando se cumplen los días previos requeridos y, en caso contrario, se cobra el porcentaje definido.

Estado	Significado	Siguiente estado
PENDIENTE	Reserva creada, sin pago completo	CONFIRMADA o CANCELADA
CONFIRMADA	Pagada en su totalidad	EN_CURSO (check-in) o CANCELADA
EN_CURSO	Check-in realizado; la habitación queda OCUPADA	FINALIZADA
FINALIZADA	Check-out realizado; la habitación vuelve a DISPONIBLE	—
CANCELADA	Cancelada con cargo o reembolso según la política	—

4. Herramientas utilizadas

Herramienta	Uso	Versión
Java (JDK)	Lenguaje del backend	21
Spring Boot [2]	Framework del backend	3.5.6
Maven [6]	Gestión de dependencias y construcción	n/a
MySQL [5]	Gestor de base de datos relacional	n/a
MySQL Workbench	Administración de la base de datos	n/a
Node.js y npm [7]	Entorno de ejecución del frontend	v24.18.0
React [3] y Vite [8]	Frontend	React 18.3.1 y Vite 5.4.11
Swagger UI (springdoc) [9]	Documentación de la API	2.8.13
Postman [10]	Pruebas de los endpoints	n/a
IDE (IntelliJ IDEA / VS Code)	Desarrollo del backend y del frontend	n/a
Git y GitHub [11, 12]	Control de versiones y repositorio	git version 2.54.0.windows.1

5. Base de datos

5.1 Diagrama relacional

La base de datos hotel_db está compuesta por 13 tablas relacionadas mediante claves foráneas. La tabla reservas es el centro del modelo, ya que conecta al huésped, la habitación, el usuario que la registra y la política de cancelación aplicable.

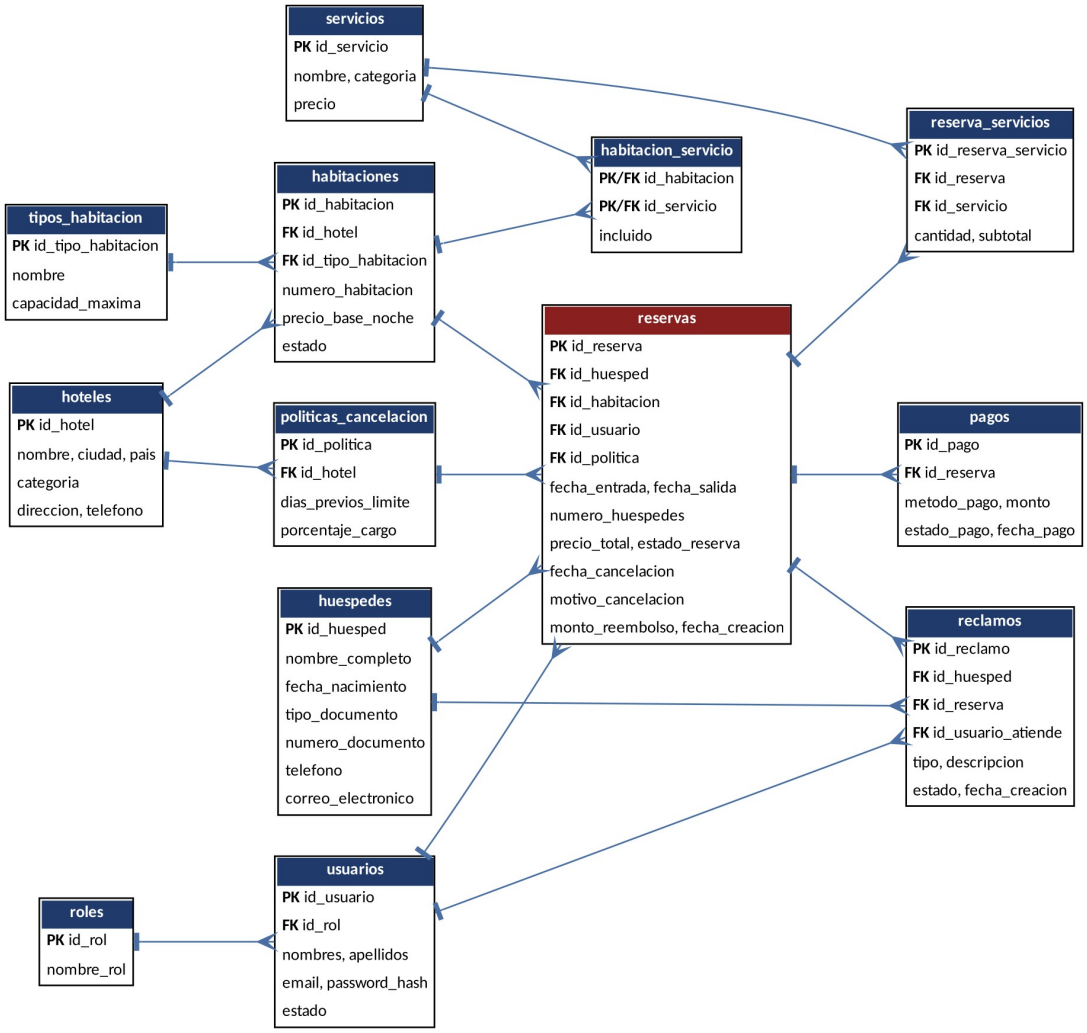


Figura 1. Modelo relacional de la base de datos hotel_db (13 tablas).

5.2 Descripción de las tablas

Tabla	Descripción
roles	Perfiles del sistema: ADMIN, RECEPCIONISTA y CLIENTE.
usuarios	Cuentas de acceso con rol, contraseña cifrada con BCrypt [13] y estado (ACTIVO, INACTIVO o BLOQUEADO).
huespedes	Datos personales y documento del huésped (DUI, pasaporte, DNI o NIT).
hoteles	Hoteles y sucursales con ciudad, país, categoría (1 a 5), dirección y teléfono.
tipos_habitacion	Tipos de habitación y su capacidad máxima.
habitaciones	Habitaciones de cada hotel con número, precio base por noche y estado (DISPONIBLE, OCUPADA o MANTENIMIENTO).
politicas_cancelacion	Días previos límite y porcentaje de cargo por cancelación, por hotel.
servicios	Catálogo de servicios (alimentación, transporte, limpieza u otros) con su precio.
habitacion_servicio	Servicios incluidos en cada habitación (relación muchos a muchos).
reservas	Reserva con huésped, habitación, usuario, política, fechas, número de huéspedes, total, estado y datos de cancelación y reembolso.
reserva_servicios	Servicios adicionales contratados en una reserva, con cantidad y subtotal.
pagos	Pagos de una reserva: método, monto, estado y fecha.

Tabla	Descripción
reclamos	Reclamos de huéspedes sobre una reserva, con tipo, estado y usuario que los atiende.

5.3 Restricciones de integridad

- Claves primarias autoincrementales y claves foráneas entre todas las tablas relacionadas.
- Restricciones UNIQUE en el correo del usuario, el número de documento del huésped, el nombre del rol y el par hotel y número de habitación.
- Restricciones CHECK: fecha de salida posterior a la de entrada, número de huéspedes y capacidad mayores que cero, precios y montos no negativos, categoría del hotel entre 1 y 5, porcentaje de cargo entre 0 y 100, y estados limitados a valores permitidos.
- El estado de la reserva admite PENDIENTE, CONFIRMADA, EN_CURSO, FINALIZADA y CANCELADA; el de pago, PENDIENTE, COMPLETADO, RECHAZADO y REEMBOLSADO.
- El script database/hotel_db.sql crea las 13 tablas y carga datos de ejemplo (3 hoteles, 4 tipos de habitación, 12 habitaciones, 6 servicios y 3 políticas de cancelación). Debe ejecutarse antes de iniciar la API; si se vuelve a ejecutar, elimina y recrea todo.
- El acceso a datos se realiza con Spring Data JPA [14] e Hibernate [15].

6. Estructura del proyecto

El repositorio se organiza en cuatro carpetas principales dentro de hotel-proyecto: backend (API REST con Spring Boot), frontend (aplicación React con Vite), database (script hotel_db.sql) y postman (colección de pruebas). Además incluye el archivo README.md y los pasos para ejecutar la API y el frontend.

```
hotel-proyecto/
|-- backend/        API REST (Spring Boot, Maven) + .env.example
|-- frontend/       Interfaz React + Vite
|-- database/       hotel_db.sql
|-- postman/        Hotel_API.postman_collection.json
`-- README.md
```

El backend usa el paquete base com.udb.hotel, organizado por capas:

Paquete	Contenido
config	Configuración de seguridad (SecurityConfig), OpenAPI e inicialización de los usuarios de prueba (DataInitializer).
controller	Controladores REST que exponen los endpoints.
service	Lógica de negocio (reservas, disponibilidad, usuarios, autenticación, reclamos y estadísticas).
repository	Acceso a datos con Spring Data JPA y consultas JPQL de disponibilidad.
entity	Entidades JPA que mapean las tablas de la base de datos.
dto	Objetos de transferencia de datos y validaciones de entrada (Bean Validation [16]).
security	Generación y validación de JWT, filtro de autenticación y detalle de usuario.
exception	Manejo global de errores con @RestControllerAdvice y respuestas JSON uniformes.
util	Reglas de validación reutilizables, fecha actual en zona de El Salvador y usuario autenticado.

El frontend (carpeta frontend/src) contiene: **api.js** (cliente HTTP con el token), **auth.jsx** (sesión), **validators.js** (validaciones), **App.jsx** (rutas protegidas por rol), **components** (Layout, CrudPage, PagoForm y componentes de interfaz) y **pages** (13 pantallas).

7. Dependencias y versiones

7.1 Backend (pom.xml)

Dependencia	Versión
spring-boot-starter-parent	3.5.6

Dependencia	Versión
spring-boot-starter-web	3.5.6 (gestionada por Spring Boot)
spring-boot-starter-data-jpa [14]	3.5.6 (gestionada por Spring Boot)
spring-boot-starter-validation	3.5.6 (gestionada por Spring Boot)
spring-boot-starter-security [17]	3.5.6 (gestionada por Spring Boot)
mysql-connector-j	Gestionada por Spring Boot 3.5.6
springdoc-openapi-starter-webmvc-ui [9]	2.8.13
jjwt-api, jjwt-impl y jjwt-jackson [18]	0.12.6
lombok [19]	Gestionada por Spring Boot 3.5.6
Java (java.version)	21

7.2 Frontend (package.json)

Dependencia	Versión
react y react-dom [3]	^18.3.1
react-router-dom [20]	^6.28.0
axios [21]	^1.7.9
vite [8]	^5.4.11
@vitejs/plugin-react	^4.3.4

Nota: jjwt 0.12.x usa la API actual (*Jwts.builder().subject()*, *Jwts.parser().verifyWith()*). Lombok se declara en *annotationProcessorPaths* del pom.xml para que compile también con JDK 23 o superior.

8. Repositorio y documentación de la API

- **URL del repositorio:** https://github.com/Dark-Side-Ghost/DFW-Sistema-Hotelero.git
- **Usuario de GitHub:** Dark-Side-Ghost
- **Documentación Swagger/OpenAPI [22]:** http://localhost:8080/swagger-ui.html (con la API en ejecución). Incluye el botón Authorize para ingresar el token JWT. La especificación en JSON está en /v3/api-docs.
- **Pruebas:** la carpeta postman contiene una colección con 74 requests y pruebas automáticas del código de respuesta esperado, incluyendo casos de error (400, 401, 403 y 409).

9. Usuarios predefinidos para pruebas

Los usuarios son creados automáticamente por la API al iniciar (DataInitializer), con la contraseña cifrada con BCrypt.

Rol	Correo	Clave
ADMIN	admin@hotel.com	Admin123*
RECEPCIONISTA	recepcion@hotel.com	Recepcion123*
CLIENTE	cliente@hotel.com	Cliente123*

Además, cualquier persona puede registrarse desde la pantalla de login; la cuenta se crea siempre con el rol CLIENTE.

10. Seguridad y control de acceso

- **Autenticación:** *POST /api/auth/login* valida el correo y la contraseña (BCrypt) y devuelve un token JWT [4] firmado con la clave secreta del servidor, con el correo y el rol del usuario y una vigencia de 8 horas. Spring Security [17] y la librería jjwt [18] se encargan de crearlo y verificarlo.
- **Sesiones sin estado:** el servidor no guarda sesiones; en cada petición el filtro JwtAuthFilter lee el encabezado *Authorization: Bearer*, verifica el token y autentica al usuario.

- **Autorización:** todos los endpoints exigen token, salvo el login, el registro y Swagger. El acceso por rol se controla con @PreAuthorize en cada controller. Si falta el token o es inválido se responde 401; si el rol no tiene permiso, 403.
- **Cuentas:** el administrador puede activar y desactivar usuarios; una cuenta inactiva no puede iniciar sesión ni usar un token anterior. Un administrador no puede desactivar ni eliminar su propia cuenta.
- **Propiedad de los datos:** un cliente solo puede consultar, modificar, pagar o cancelar sus propias reservas.
- **Secretos fuera del repositorio:** la clave JWT y las credenciales de MySQL se leen de variables de entorno o del archivo local backend/.env, que no se sube a GitHub (ver sección 14).

11. Validaciones y manejo de errores

Las validaciones se implementan en el servidor con Bean Validation y reglas en los servicios; el frontend las repite para dar respuesta inmediata, pero la validación definitiva es siempre la del servidor.

Tipo	Ejemplos implementados
Campos vacíos	Nombre del hotel, fechas de la reserva, método de pago, motivo de cancelación, datos del huésped.
Tipo de dato y formato (expresiones regulares)	Nombres solo con letras; teléfono (8 a 15 dígitos); DUI con formato 00000000-0; correo; contraseña de 8 a 64 caracteres con letra y número; estrellas y capacidades enteras; precios con máximo 2 decimales.
Congruencia con la realidad	Fecha de entrada no anterior a hoy; salida posterior a la entrada; estadía máxima de 30 noches; huésped titular mayor de 18 años; huéspedes no mayores que la capacidad; habitación sin reservas que se crucen; tarjeta no vencida; no pagar una reserva ya pagada.

Todas las respuestas de error tienen el mismo formato JSON, con un mensaje amigable y, cuando corresponde, el detalle por campo:

```
{
  "timestamp": "2026-10-29T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Hay datos inválidos o incompletos. Revisa los campos indicados.",
  "ruta": "/api/hoteles",
  "errores": { "nombre": "El nombre del hotel es obligatorio" }
}
```

Código	Uso
200, 201, 204	Operación correcta, recurso creado, eliminado sin contenido
400	Datos inválidos o regla de negocio incumplida
401	Sin sesión, token vencido o credenciales incorrectas
403	El rol no tiene permiso o la reserva pertenece a otro usuario
404	Recurso inexistente
409	Duplicado o conflicto (correo repetido, habitación ya reservada en esas fechas)
500	Error inesperado (el detalle solo queda en el log del servidor)

12. Endpoints principales

Módulo	Endpoints	Acceso
Autenticación	POST /api/auth/login · POST /api/auth/registro	Público
Usuarios	GET, POST /api/usuarios · GET, PUT, DELETE /{id} · PATCH /{id}/estado	ADMIN
Hoteles, tipos, servicios y políticas	GET, POST, PUT, DELETE en /api/hoteles, /api/tipos-habitacion, /api/servicios y /api/politicas	Lectura: autenticado · Escritura: ADMIN
Habitaciones	CRUD en /api/habitaciones · GET /api/habitaciones/disponibles	Lectura y búsqueda: autenticado · Escritura: ADMIN

Módulo	Endpoints	Acceso
Reservas	POST /api/reservas · GET /mias · GET /{id} · PUT /{id} · POST /{id}/pagos · POST /{id}/cancelar	Autenticado (dueño o personal)
Reservas (personal)	GET /api/reservas · GET /hoy · POST /{id}/check-in · POST /{id}/check-out	ADMIN, RECEPCIONISTA
Reclamos	POST /api/reclamos · GET /mios · GET /api/reclamos · PUT /{id}/estado · DELETE /{id}	Cliente · Personal · ADMIN (eliminar)
Estadísticas	GET /api/estadisticas	ADMIN

13. Frontend

El frontend es una aplicación de una sola página creada con React [3] y Vite [8]. Usa React Router [20] para la navegación y axios [21] para consumir la API. Las rutas están protegidas por sesión y por rol, y el menú cambia según el perfil del usuario.

Rol	Pantallas
ADMIN	Estadísticas, hoteles, habitaciones, tipos de habitación, servicios, políticas de cancelación, usuarios, reservas, reclamos y nueva reserva.
RECEPCIONISTA	Reservas del día (pagos, check-in, check-out y cancelación), reclamos y nueva reserva.
CLIENTE	Buscar y reservar (flujo de seis pasos: buscar, elegir habitación, datos del huésped, resumen, pago y confirmación) y mis reservas (pagar, modificar, cancelar y reportar un problema).

Durante el desarrollo, Vite redirige las peticiones que comienzan con */api* hacia *http://localhost:8080* (proxy configurado en vite.config.js), por lo que el navegador trabaja con un solo origen. El token JWT se guarda en el navegador y un interceptor de axios lo agrega a cada petición; ante una respuesta 401 la aplicación regresa al inicio de sesión. Los mensajes de error de la API se muestran directamente al usuario.

14. Configuración y ejecución del proyecto

14.1 Archivo .env y variables de configuración

El repositorio no contiene contraseñas reales. Las credenciales se definen en un archivo local **backend/.env** (que está en .gitignore) creado a partir de la plantilla **backend/.env.example**. Cada variable va en una sola línea, sin espacios ni comillas:

```
DB_USER=root
DB_PASSWORD=la_clave_de_su_mysql
JWT_SECRET=una_clave_aleatoria_de_40_caracteres_o_mas
```

Variable	Descripción	Valor por defecto
DB_USER	Usuario de MySQL	root
DB_PASSWORD	Contraseña de MySQL	root
JWT_SECRET	Clave para firmar los tokens JWT (mínimo 32 caracteres)	Clave de desarrollo pública incluida en application.properties
JWT_EXPIRATION_MS	Duración del token en milisegundos	28800000 (8 horas)

Las mismas variables pueden definirse como variables de entorno del sistema. El archivo .env se lee desde la carpeta donde se inicia la API, por lo que se recomienda ejecutar el backend desde la carpeta backend.

14.2 Pasos de ejecución

- Importar el script database/hotel_db.sql en MySQL (una sola vez, antes de iniciar la API).
- Crear backend/.env con los valores propios (sección 14.1). Si la contraseña de MySQL es root, este paso es opcional.
- Iniciar el backend desde la carpeta backend con *mvn spring-boot:run*. La API queda en http://localhost:8080.
- En la carpeta frontend ejecutar *npm install* y luego *npm run dev*. La aplicación queda en http://localhost:5173.

- Iniciar sesión con alguno de los usuarios de la sección 9.
- Opcional: importar la colección de postman, ejecutar primero la carpeta “0. Login” y luego el resto de las carpetas.

15. Referencias

[1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[2] VMware, Inc. (Broadcom), “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: https://docs.spring.io/spring-boot/. [Accessed: Oct. 29, 2026].

[3] Meta Platforms, Inc., “React – The library for web and native user interfaces,” react.dev. [Online]. Available: https://react.dev/. [Accessed: Oct. 29, 2026].

[4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Eng. Task Force (IETF), RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[5] Oracle Corporation, “MySQL 8.0 Reference Manual,” dev.mysql.com. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. [Accessed: Oct. 29, 2026].

[6] The Apache Software Foundation, “Apache Maven Project,” maven.apache.org. [Online]. Available: https://maven.apache.org/. [Accessed: Oct. 29, 2026].

[7] OpenJS Foundation, “Node.js documentation,” nodejs.org. [Online]. Available: https://nodejs.org/docs/latest/api/. [Accessed: Oct. 29, 2026].

[8] VoidZero Inc., “Vite – Next generation frontend tooling,” vite.dev. [Online]. Available: https://vite.dev/. [Accessed: Oct. 29, 2026].

[9] springdoc.org contributors, “springdoc-openapi: OpenAPI 3 Library for Spring Boot,” springdoc.org. [Online]. Available: https://springdoc.org/. [Accessed: Oct. 29, 2026].

[10] Postman, Inc., “Postman Docs,” learning.postman.com. [Online]. Available: https://learning.postman.com/docs/. [Accessed: Oct. 29, 2026].

[11] S. Chacon and B. Straub, Pro Git, 2nd ed. New York, NY, USA: Apress, 2014. [Online]. Available: https://git-scm.com/book/

[12] GitHub, Inc., “GitHub Docs,” docs.github.com. [Online]. Available: https://docs.github.com/. [Accessed: Oct. 29, 2026].

[13] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annu. Tech. Conf. (FREENIX Track), Monterey, CA, USA, Jun. 1999, pp. 81–91.

[14] VMware, Inc. (Broadcom), “Spring Data JPA – Reference Documentation,” Spring.io. [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/. [Accessed: Oct. 29, 2026].

[15] Red Hat, Inc., “Hibernate ORM,” hibernate.org. [Online]. Available: https://hibernate.org/orm/documentation/. [Accessed: Oct. 29, 2026].

[16] Jakarta EE, “Jakarta Bean Validation Specification,” Eclipse Foundation. [Online]. Available: https://beanvalidation.org/. [Accessed: Oct. 29, 2026].

[17] VMware, Inc. (Broadcom), “Spring Security Reference,” Spring.io. [Online]. Available: https://docs.spring.io/spring-security/reference/. [Accessed: Oct. 29, 2026].

[18] Les Hazlewood and contributors, “JJWT – Java JWT: JSON Web Token for Java and Android,” GitHub. [Online]. Available: https://github.com/jwtk/jjwt. [Accessed: Oct. 29, 2026].

[19] The Project Lombok Authors, “Project Lombok,” projectlombok.org. [Online]. Available: https://projectlombok.org/. [Accessed: Oct. 29, 2026].

[20] Remix Software Inc. / Shopify, “React Router documentation,” reactrouter.com. [Online]. Available: https://reactrouter.com/. [Accessed: Oct. 29, 2026].

[21] M. Zabriskie and contributors, “Axios – Promise based HTTP client for the browser and Node.js,” axios-http.com. [Online]. Available: https://axios-http.com/. [Accessed: Oct. 29, 2026].

[22] OpenAPI Initiative (Linux Foundation), “OpenAPI Specification,” spec.openapis.org. [Online]. Available: https://spec.openapis.org/oas/latest.html. [Accessed: Oct. 29, 2026].